

Next → ⚠ Error Handling

What happens when:

Google PageSpeed API



API fails

We don't want the whole application exploding into some

Show more ▾

Absolutely 👍 Let's continue **Next.js Lesson 4 → Error Handling**.

We'll keep this tied to your **Web Performance Analyzer**, so you understand not just *what* Next.js gives you, but **where and why you would actually use it**.

⚠ Lesson 4 — Error Handling

Imagine your analyzer does this:

User enters URL



Next.js Server



Google PageSpeed API



💥 API fails



There are actually **two different problems** here:

1. **What should happen when a component/page crashes?**
2. **What should happen when our API request fails?**

Next.js gives us mechanisms for both.

1. `error.tsx`

Next.js has a special file:

error.tsx



It creates an **Error Boundary** for a route segment.

For example:

```
app/  
├─ layout.tsx  
├─ page.tsx  
├─ error.tsx  
└─ analyzer/  
    ├─ page.tsx  
    └─ error.tsx
```



Think of it like this:

```
app/  
├─ page.tsx  
├─ error.tsx      ← catches errors in this segment  
└─ analyzer/  
    ├─ page.tsx  
    └─ error.tsx  ← catches errors in analyzer segment
```



What does an Error Boundary do?

Suppose this happens:

```
export default function AnalyzerPage() {  
  throw new Error("Something went wrong");  
  
  return <div>Analyzer</div>;  
}
```

Normally, that would cause the rendering to fail.

But if we have:

```
// app/analyzer/error.tsx  
  
"use client";
```

```
export default function Error({
  reset,
}: {
  error: Error & { digest?: string };
  reset: () => void;
}) {
  return (
    <div>
      <h2>Something went wrong.</h2>

      <button onClick={() => reset()}>
        Try again
      </button>
    </div>
  );
}
```

Next.js can show:

Something went wrong.



[Try again]

instead of the user getting some horrible broken UI.

2. Why is `error.tsx` a Client Component?

This is an important interview/detail question.

Notice:

```
"use client";
```

Why?

Because the error UI needs to be **interactive**.

Specifically:

```
<button onClick={() => reset()}>
```

`onClick` requires client-side JavaScript.

So:

```
error.tsx
  ↓
Client Component
  ↓
can respond to user interaction
  ↓
reset()
```



The rest of your route can still be Server Components.

That's another example of Next.js allowing you to keep most of your application server-side while making **only the interactive pieces client-side**.

3. What exactly does `reset()` do?

This is where people often misunderstand it.

Suppose:

```
Analyzer page
  ↓
API request
  ↓
🔥
```



The error boundary appears:

```
Something went wrong.

[ Try again ]
```



When the user clicks:

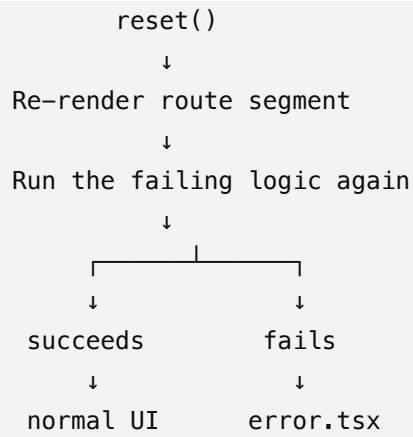
```
reset()
```

Next.js attempts to **re-render that route segment**.

Conceptually:

```
User clicks Try again
  ↓
```





So `reset()` doesn't magically fix the API.

It basically says:

"Try rendering this route again."

4. But here's the important distinction 🚩

For your analyzer, we **shouldn't necessarily throw an error for every PageSpeed API failure.**

For example:

```
Google API
  ↓
returns 400
  ↓
Invalid URL
```



That's not really an application crash.

It's an **expected error**.

The user entered something invalid.

So we'd rather show:

```
⚠️ Unable to analyze this URL.

Please enter a valid website URL.
```



rather than:

Something went wrong.



[Try again]

This gives us an important rule:

Expected errors should usually be handled explicitly. Unexpected errors should be caught by error boundaries.

That's a VERY useful distinction.

5. API failure in your analyzer

Imagine your server-side function:

```
const response = await fetch(  
  `https://pagespeedonline.googleapis.com/...`  
);
```

What if Google returns:

500 Internal Server Error



or:

429 Too Many Requests



or:

401 Unauthorized



You shouldn't just blindly do:

```
const data = await response.json();
```

You should check:

```
const response = await fetch(...);

if (!response.ok) {
  throw new Error("PageSpeed API request failed");
}

const data = await response.json();
```

Now you've deliberately turned the unexpected API failure into an error your application can handle.

6. Where does that error go?

This depends on **where the request happens**.

For example:

```
export default async function AnalyzerPage() {
  const data = await getPageSpeedData();

  return <Results data={data} />;
}
```

And:

```
async function getPageSpeedData() {
  const response = await fetch(...);

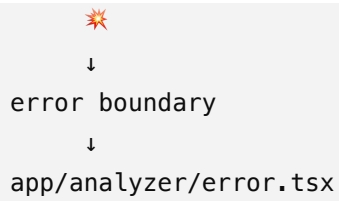
  if (!response.ok) {
    throw new Error("PageSpeed API failed");
  }

  return response.json();
}
```

If that error isn't handled locally:

```
AnalyzerPage
  ↓
getPageSpeedData()
  ↓
fetch()
  ↓
```





```
graph TD; A[Star Icon] --> B[error boundary]; B --> C[app/analyzer/error.tsx]
```

So:

app/analyzer/error.tsx

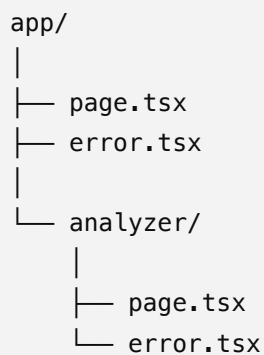


can provide the fallback UI.

7. Route-level error boundaries

This is where the folder structure becomes powerful.

Suppose:



```
graph TD; app[app/] --- page1[page.tsx]; app --- error1[error.tsx]; app --- analyzer[analyzer/]; analyzer --- page2[page.tsx]; analyzer --- error2[error.tsx]
```



An error inside:

/analyzer



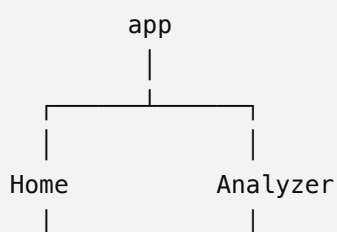
can be caught by:

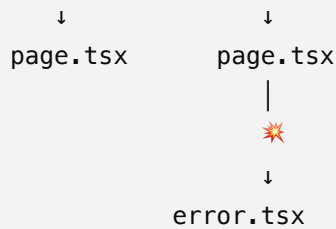
/analyzer/error.tsx



while an error in another part of the application can be handled by its relevant boundary.

Conceptually:





This means you don't have to turn your **entire application** into an error state just because one route has a problem.

8. What about `global-error.tsx` ?

There's one more special file worth knowing:

`global-error.tsx`



This is for errors that aren't handled by the normal route-level boundaries, particularly errors involving the root layout.

Conceptually:

```
error.tsx
  ↓
route-level recovery
```



while:

```
global-error.tsx
  ↓
global fallback
```



For your project, you probably won't need to spend much time on this.

Know that it exists; understand `error.tsx` much more deeply.

That's enough for project/interview purposes.

9. Now: `notFound()`

This is a **different concept**.

Suppose your application has:

/analyzer/abc123



and that analysis doesn't exist.

That's not necessarily an application error.

It's:

```
"This resource doesn't exist."
```

Next.js provides:

```
notFound();
```

from:

```
import { notFound } from "next/navigation";
```

For example:

```
import { notFound } from "next/navigation";

export default async function AnalysisPage() {
  const analysis = await getAnalysis();

  if (!analysis) {
    notFound();
  }

  return <Results analysis={analysis} />;
}
```

Then you can have:

```
app/
├── analyzer/
│   ├── [id]/
│   │   ├── page.tsx
│   │   └── not-found.tsx
```



And:

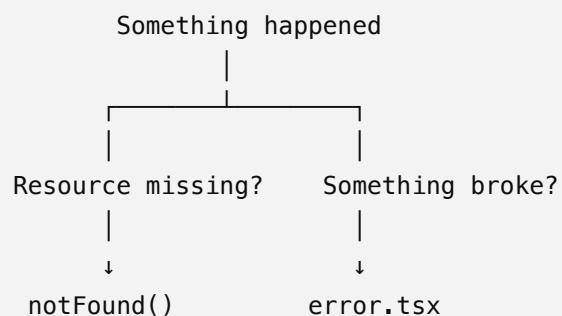
```
export default function NotFound() {  
  return (  
    <div>  
      <h2>Analysis not found</h2>  
      <p>We couldn't find this analysis.</p>  
    </div>  
  );  
}
```

10. `error.tsx` vs `notFound()`

This distinction is **very important**.

Situation	Use
Something unexpectedly crashes	<code>error.tsx</code>
API/server operation unexpectedly fails	<code>error.tsx</code>
Resource doesn't exist	<code>notFound()</code>
User requests an invalid/nonexistent resource	<code>notFound()</code>
Expected validation problem	Handle explicitly

Think:



Apply this to YOUR analyzer

Imagine the user enters:

`https://example.com`

Case 1 — Successful request

```
URL
↓
PageSpeed API
↓
200 OK
↓
Results
```



Normal UI.

Case 2 — Invalid URL

```
"hello"
↓
validation fails
↓
Show:
"Please enter a valid URL"
```



No `error.tsx`.

It's an **expected user input problem**.

Case 3 — PageSpeed API temporarily fails

```
URL
↓
PageSpeed API
↓
500
↓
error
↓
error.tsx
```



Show something like:

```
We couldn't analyze this website right now.

[ Try again ]
```



Case 4 — Analysis ID doesn't exist

```
/analyzer/12345
  ↓
database
  ↓
nothing found
  ↓
notFound()
  ↓
Analysis not found
```

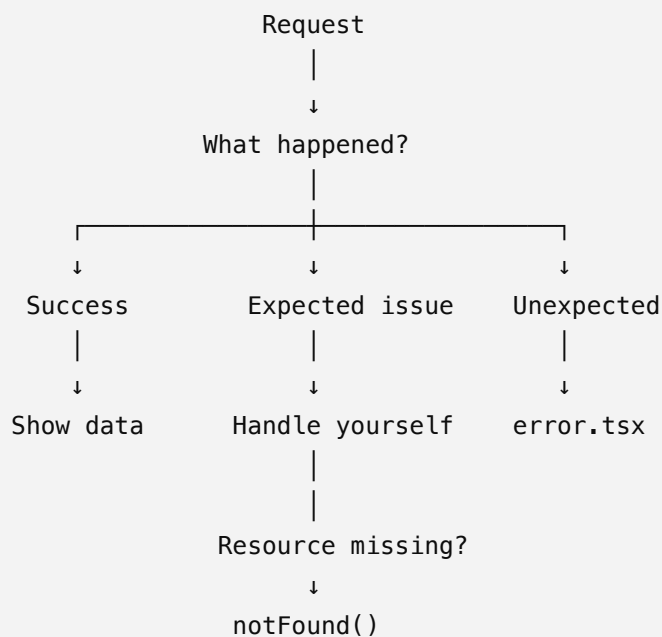


That's **404 behavior**, not an application crash.

The mental model I want you to remember

Don't memorize random Next.js filenames.

Think in terms of **what happened**:



And that is the real purpose of Next.js error handling:

Keep failures localized and turn them into useful UI instead of letting one failure destroy the user experience. 💡

One subtle point before we move on

You might be wondering:

"If `error.tsx` catches API errors, why don't we just throw an error for everything?"

Because **throwing is for exceptional/unexpected failures**.

If the user simply typed:

abc



that's not an exceptional server failure.

You want:

Please enter a valid URL.



not:

🔥 ERROR BOUNDARY ACTIVATED



That distinction becomes even more important when we build your analyzer.

Next up: `loading.tsx` → then **Streaming + Suspense**. After that we'll do the **final Lesson 4 architecture exercise + recap**, and I'll explicitly mark:

🚩 **LESSON 4 COMPLETE**



so wht come under expected what come under
unexpected what come under not found

Yep — this is the **key distinction**. Once this clicks, `error.tsx` becomes much easier.



Think of the three categories like this:

Category	Meaning	Example in your analyzer	What to do
 Expected	We <i>know this can happen</i> and can explain it to the user	Invalid URL, missing required input, PageSpeed says URL is invalid	Handle it yourself
 Unexpected	Something <i>broke</i> that normally shouldn't break	Server bug, unexpected exception, API crashes unexpectedly	<code>error.tsx</code>
 Not Found	The requested resource simply <i>doesn't exist</i>	<code>/analysis/123</code> but analysis <code>123</code> doesn't exist	<code>notFound()</code>

1. Expected errors

These are **normal situations in your application's flow**.

For example:

User enters:
"hello"



Your application checks it:

```
if (!isValidUrl(url)) {  
  // show validation message  
}
```

You might show:

 Please enter a valid website URL.



That's expected.

Other examples

User didn't enter URL
↓
"URL is required"



PageSpeed API says:
"This URL cannot be analyzed"



↓
"Please check the URL and try again"

User exceeds your allowed analysis limit



↓
"You've reached today's limit"

These aren't really **application crashes**. They're situations your application is designed to handle.

So: don't throw them into `error.tsx` just because they're technically errors.

2. Unexpected errors

These are things that **shouldn't normally happen**.

For example, your code has:

```
const data = await getPageSpeedData();
```

and something completely unexpected happens:

Your code



↓

PageSpeed API

↓

🔥 server exception

Or perhaps:

```
const result = data.performance.score;
```

But unexpectedly:

```
data = null
```



and your code crashes.

Or there's a server/database bug:

Database connection failed



These are things where you basically say:

"Uh-oh. Something went wrong on our side."

That's when:

error.tsx



is useful.

The user gets something like:

🙄 Something went wrong.



Please try again.

[Try again]

🟡 3. Not Found

This one is different.

There isn't really a **failure**.

The user asked for something that **doesn't exist**.

Imagine your analyzer saves reports:

/analyzer/abc123

/analyzer/xyz789



Someone visits:

/analyzer/does-not-exist



Your database says:

✖ No analysis with this ID



That's **not an unexpected crash**.

It's simply:

"We looked for it. It's not there."

So:

```
if (!analysis) {  
  notFound();  
}
```

And your:

not-found.tsx



can show:

 Analysis not found



This analysis may have been deleted
or the URL may be incorrect.

[Back to Analyzer]



The easiest way to decide

Ask yourself **one** question:

"Did something break, or did we simply get an answer we expected?"

Example 1

User enters:

banana



Your app says:

That's not a valid URL.



Expected

Example 2

User visits:

```
/analyzer/123
```



You search for analysis `123` .

Nothing exists.

 **Not Found**

Example 3

User enters a perfectly valid URL.

Your server calls Google.

Google suddenly returns a server failure and your operation throws.

 **Unexpected**

Example 4

Your code has a bug:

```
result.performance.score.toFixed(2)
```

but `performance` unexpectedly doesn't exist.

 **Unexpected**

One more important nuance

A PageSpeed API failure isn't automatically an `error.tsx` situation.

It depends on **why** it failed.

For example:

```
PageSpeed
```

↓



"Invalid URL"

That's an expected outcome → handle it yourself.

But:

PageSpeed

↓

500 Internal Server Error



That's an unexpected service failure → you might throw → `error.tsx`.

And:

Your own database

↓

Analysis ID doesn't exist



That's → `notFound()`.

So don't classify based merely on:

"Did the API return an error status?"

Instead classify based on **what the situation means to your application**.

Your analyzer's cheat sheet

User input problem

↓

● EXPECTED

Handle yourself

Requested resource doesn't exist

↓

● NOT FOUND

`notFound()`

Something unexpectedly broke

↓

● UNEXPECTED

`error.tsx`



That's the mental model. Once you have that, the actual Next.js APIs are just the implementation.



ChatGPT can make mistakes. Check important info.